Either way, we still need to define what highest priority means. If we assume that highest priority means a larger number, i.e. $p_1 > p_2$ means p_1 is the highest priority, then we call our heap a *max-heap*.

Other times we need to return smallest numbers first, and so we assume instead $p_1 > p_2$ means p_2 is the highest priority. In this case, we are using a *min-heap*.

In the rest of the book, and in particular in the coding section, we assume we are implementing a min-heap.

The implementation of a min-heap differs only slightly from a max-heap, code is pretty much symmetric, and you just need to exchange all occurrences of $<$ and $\leq$ to $>$ and $\geq$ respectively, and swap `min` with `max`.

Or, alternatively, it's even easier, if you have an implementation of one, get the other by simply taking the reciprocal of priorities (either in your priority function or when you pass priority explicitly).

To give you a concrete example, suppose you have a min-heap that you use to store `(age, task)` pairs: a min-heap will return the task with smallest ages first. Since we might instead be interested in having oldest task returned first, we would rather need a max-heap: turns out we can get the same result without changing any code for out heap! We just need to store elements as tuples `(-age, task)` instead. If `x.age < y.age`, in fact, then `-x.age > -y.age`, and thus the min-heap will return first the tasks whose age has the largest absolute value.